Lab 5: Static Code Analysis

Smart Home Device Controller (smart_home.py)

Name: Vismay Chandra Dev
SRN: PES2UG24CS597
Section: 5J
Repository: github.com/VismayCD/L5_36_smart_home

1. Results

I ran Pylint, Bandit and Flake8 on `smart_home.py` before changing anything. Then I fixed the issues in three separate commits, one for each tool, and ran all three tools again at the end.

Tool	Before	After
Pylint	4.80 / 10 (26 messages)	10.00 / 10
Bandit	4 issues: 1 High, 1 Medium, 2 Low	No issues identified
Flake8	4 issues	0 issues

#	Commit	Issues fixed (see table below)
1	Fix security issues reported by Bandit	1, 3, 4, 8
2	Fix code quality issues reported by Pylint	2, 5, 6, 7, 9, 10, 11
3	Fix PEP 8 style issues reported by Flake8	12, 13, 14, 15

2. Issue Documentation

Line numbers are from the original file. The high and medium severity issues are listed first.

#	Issue	Type	Line(s)	Description	Fix approach
1	**Shell command injection** Bandit B602 · High	Security	51	`runScene` stuck the script name onto `"bash scenes/"` and ran it with `shell=True`. A name like `x.sh; rm -rf ~` would have run a second command.	It only accepts a file that is actually inside `scenes/`, finds bash with `shutil.which` and runs it as a list of arguments with no shell.
2	**Mutable default argument** Pylint W0102	Bug	11–13	`settings={}` is created once and shared by every device. That's why the demo said the ceiling light had a temperature of 30.	Default is None now and every device gets its own new dict. The light prints `{}`.
3	**Binding to all interfaces** Bandit B104 · Medium	Security	4	`HUB_HOST = "0.0.0.0"` opens the hub on every network interface. It also isn't an address an app can connect to.	Defaults to `127.0.0.1`, and can be changed through the `SMART_HOME_HUB_HOST` environment variable.
4	**Hard-coded password** Bandit B105 · Low	Security	7	The hub password was written in the code, so it also ends up in the git history.	Read from the `SMART_HOME_HUB_PASSWORD` environment variable instead.
5	**Inconsistent returns** Pylint R1710	Bug	16–20	`toggleDevice` returned True/False normally but nothing at all when the device didn't exist.	Returns None explicitly for an unknown device.

#	Issue	Type	Line(s)	Description	Fix approach
6	**Crash on unknown device** Found by reading the code	Bug	24	`setTemperature` used `devices[name]` directly, so a typo in the name threw a `KeyError`.	Uses `.get()`, prints "Device not found" and returns `False`, the same way toggle does.
7	**No file encoding** Pylint W1514	Portability	41	`open()` without an encoding uses the OS default, so Windows and Linux could write the JSON differently.	Added `encoding="utf-8"`.
8	**subprocess import / call** Bandit B404 · Low, B603	Security review	2, 51	Bandit flags any use of `subprocess` so that someone checks it by hand.	Checked it after fix 1, then marked it `# nosec B404` / `# nosec B603` with a comment explaining why.
9	**camelCase names** Pylint C0103 · 13 messages	Style	11–49	Functions and arguments were camelCase: `addDevice`, `deviceName`, `fileName`, and so on.	Renamed everything to snake_case. Also renamed `devices` to `DEVICES` to keep it consistent.
10	**Missing docstrings** Pylint C0114, C0116 · 10 messages	Docs	1, 11–54	No module docstring and no docstring on any of the 9 functions.	Added a short docstring to the module and to each function.
11	**Demo runs on import** Found by reading the code	Design	72	`main()` was called at the bottom of the file, so just importing it ran the demo and wrote `home_state.json`.	Moved it under `if __name__ == "__main__":`.
12	**Space before colon** Flake8 E203	Style	6	`{"light" : 10, ...}` has a space before the colon.	Removed the space.
13	**Missing blank line** Flake8 E302	Style	40	Only one blank line before `def saveState`.	Added the second blank line.
14	**Inline comment spacing** Flake8 E261	Style	66	Only one space before `# for app setup`.	Changed it to two spaces.
15	**Line too long** Flake8 E501	Style	68	The line was 80 characters; the limit is 79.	Put the list comprehension in a `devices_on` variable first, then printed that.

3. Reflection

1. Which issues were the easiest to fix, and which were the hardest? Why?

The Flake8 ones were the easiest. Every line of the report gives you the exact line and column, and the fix is obvious once you look: delete a space, add a blank line, split a long line. None of them can break the program, so there was nothing to think about. The docstrings and the renaming were simple too, just a bit tedious, because after renaming the functions I had to update every call in `main()`.

The hardest one was the `shell=True` problem in `runScene`. My first idea was to just delete `shell=True`, but that on its own doesn't make it safe. I had to work out what input should be allowed in the first place. I settled on "a file that's actually inside the scenes folder", so `../` paths and names with a semicolon get rejected. Even after that, Bandit still complained about `subprocess`, and I had to decide whether that was a real problem or just Bandit asking me to double-check.

The mutable default argument was tricky in a different way. The program ran without any error, but the output said the ceiling light had a temperature of 30, which makes no sense for a light. It took me a minute to trace that back to `settings={}` and remember that Python creates default values once, when the function is defined, not every time it's called.

2. Did the static analysis tools report any false positives? If so, describe one example.

Yes. The clearest one is Bandit's B404, which fires just because the file imports `subprocess`. Even after I rewrote `runScene` to pass a fixed list of arguments with no shell, Bandit still flagged the import, and the new call too (B603). Those warnings are really there to make someone review the code, not because something is wrong. Once I had checked it, I suppressed them with `# nosec B404` and `# nosec B603` and wrote a comment saying why. I used the specific test IDs instead of a bare `# nosec`, so if a different problem ever shows up on those lines, Bandit will still report it.

The hard-coded password warning (B105) is more of a borderline case. "smarthome-demo" is obviously just a demo value, but Bandit is right that passwords shouldn't be in the code at all, so I moved it to an environment variable anyway.

3. How would you integrate static analysis tools into your actual software development workflow?

Running the tools by hand once at the end, the way I did in this lab, isn't how I'd want to do it on a real project. I'd set them up in three places:

- **In the editor:** Pylint and Flake8 both have VS Code extensions, so problems get underlined while I type. It's much easier to fix a name right when I write the function than to rename ten things later.
- **Before every commit:** a pre-commit hook that runs flake8, pylint and bandit and blocks the commit if something new shows up, so bad code never reaches the repo.
- **In CI:** a GitHub Actions workflow that runs all three on every push and pull request, with `pylint --fail-under=9.5` and Bandit set to fail on medium or high severity. If the check fails, the PR can't be merged.

I'd also keep the rules in a shared config file (`pyproject.toml` or `.flake8`) so everyone on the team checks against the same settings. On something like our SE mini-project, where four of us push to the same repo and everyone writes code slightly differently, this would have saved a lot of back and forth.

4. What tangible improvements did you observe in the code quality, readability, or potential robustness after applying the fixes?

The biggest one is that the program is actually correct now. Before, every device shared one settings dictionary, so setting the thermostat also changed the light. Now each device has its own settings, `set_temperature` doesn't crash on a name it doesn't know, and `toggle_device` always returns something sensible.

On the security side, `run_scene` can't be used to run arbitrary shell commands anymore, the password isn't in the code, and the hub only listens on localhost unless you configure it otherwise.

It's also easier to read. All the names are snake_case, every function has a docstring, `MIN_TEMPERATURE` and `MAX_TEMPERATURE` replace the bare 16 and 30, and the spacing follows PEP 8. The `__main__` guard means I can import the file to test it without it running the demo and writing a JSON file each time.

In numbers: Pylint went from 4.80 to 10.00, Bandit from 4 issues (one of them High severity) to none, and Flake8 from 4 issues to 0. Now that everything is clean, a new warning will stand out right away instead of getting lost among thirty old ones.